

Advanced Programming II

Official exam. June, 23

Course 2024/25

PART 1: Functional Block

Exercise 1.1 (4 points)

Let's assume the following trait with the basic operations of directed graphs.

```
trait Graph[N] {  
  def addNode(node: N): Graph[N] // adds a node to the graph  
  def addEdge(from: N, to: N): Graph[N] // adds an edge from the from node to the to node, throws a  
    IllegalArgumentException if any of the nodes do not exist in the graph.  
  def neighbors(node: N): Set[N] // returns the neighbouring nodes of a given node (endpoints of edges  
    where the Node node is the source)  
  def hasNode(node: N): Boolean // checks if the graph contains a node  
}
```

You have to develop a `GraphOverMap[T]` class that implements `Graph[T]`. The class must use a `Map[T, Set[T]]` to store the nodes and edges of the graph. If a graph has an edge `a -> b`, the corresponding map will have `a` as the key and `b` will be in the set of elements associated with `a`. In addition to the trait methods, the `GraphOverMap[T]` class will include:

- A private main constructor that takes an argument of type `Map[T, Set[T]]`.
- A constructor without arguments (`this()`).
- Redefinitions of the `toString` methods (showing the set in the format "`Graph(1 -> 2, 2 -> 3, 3 -> 1, 3 -> 2)`"), `hashCode` and `equals` to ensure that two maps with the same nodes and edges are considered the same.

Exercise 1.2 (2 points)

Using `match` and higher-order functions, define a `combinations` function that receives a `k`-number and a list, and returns all `k`-size combinations of the items in the list. For example:

```
combinations(2, List(1, 2, 3, 4)) is List(List(1, 2), List(1, 3), List(1, 4), List(2, 3), List(2, 4), List(3, 4))  
combinations(3, List(1, 2, 3, 4)) is List(List(1, 2, 3), List(1, 2, 4), List(1, 3, 4), List(2, 3, 4))
```

Exercise 1.3 (2 points)

Given the class

```
class Person(name: String, age: Int) {  
  def getAge = age  
  def getName = name  
  override def toString: String = s"Person($name, $age)"  
}
```

and a value

```
val people = List(Person("Alice", 30), ..., Person("Oscar", 36))
```

- Create a map in which the keys are the initial letters of the names and the values are sets of the names of the people with such an initial
- Create a map in which each person is associated with their age in months.
- Calculate the average age of people.

Exercise 1.4 (2 points)

Without using the `partition` function, you implement a `partitionBy` function that partitions a list into two lists based on a predicate, that is, it puts the items that satisfy the predicate in one list and those that do not in another. For example:

```
partitionBy(List(1, 2, 3, 4, 5, 6))(_ % 2 == 0) // Output: (List(2, 4, 6), List(1, 3, 5))
```

```
partitionBy(List("apple", "banana", "cherry", "date"))(_.length > 5) // Output: (List(banana, cherry), List(apple, date))
```

List API Functions	Explanation
<code>list.splitAt(n:Int)</code>	Split list in two by the position <code>n</code> and return the two sublists (<code>List(0..n-1)</code> , <code>List(n.. list.size-1)</code>)
<code>list.filter(p)</code>	Returns the list sublist with the items that satisfy <code>p</code>
<code>list.map(f)</code>	Returns the list obtained by applying <code>f</code> to each list item
<code>list.foldLeft(z)(f)</code> <code>list.foldRight(z)(f)</code>	Apply the binary operation <code>f</code> to list items from left to right (or from right to left) starting with <code>z</code>
<code>list.sum</code>	Sum up the list items
<code>list1.zip(list2)</code>	Returns the list of pairs constructed with elements from <code>list1</code> and <code>list2</code>
<code>list.groupBy(f)</code>	Build a map from the list items using the <code>f</code> function

Note: If you use REPL, remember that to load the functions of a file into REPL you use the **command** `:load <path>` where `<path>` is the path to the file.

Concurrency block

The Game of Musical Chairs

To play the game of Musical Chairs, we place several chairs in a circle, with the seat facing outwards. If we have `J` players, we will have `J - 1` chairs. For example, if there are 5 players there should be 4 chairs. Initially, the players form a circle around the chairs, and then the music starts playing. At that moment the participants begin to dance walking around the chairs. When the music stops, players must sit in one of the chairs. Because there are less chairs than players, one of the players will remain standing, being eliminated. A chair is then removed and the game is repeated, continuing round after round in the same way until only one chair and two players remain. The player who manages to sit in that last round wins the game. For example, if there are initially 4 chairs and 5 players, the games have the following configurations:

Iter. 0: 4 chairs-5 students
Iter. 1: 3 chairs-4 students
Iter. 2: 2 chairs-3 students
Iter. 3: 1 chair-2 students. The student who sits wins.

PART 2: Semaphores block

Exercise 2.1. (7 points) Implement the game of Musical Chairs using **binary semaphores**. To do so, you must use the skeleton provided in the VC which contains the following classes and functions:

- The **MusicalChairs** class has as a parameter the number of initial chairs in the game (those of the first stage) and provides the following functions:
 - **def iSit(id: Int, p: Int): Boolean**. The id player uses it to try to sit on one of the chairs in the game's p-game. The function returns true if the player manages to sit down and false, otherwise.
 - **def waitCongratulations(): Unit** and **def congratulates(id: Int)** are asked for in exercise 2.2.
 - The main function that creates the object **MusicalChairs** and the player threads whose behaviour is already implemented. Each thread calls the iSit function in successive stages of the game until it either loses (the function returns false) or wins the game (iSit returns true in the last stage).

The following synchronization conditions must be implemented in the iSit function:

CS-Player-1: A player who manages to sit in the p stage must sit down until all the other players try to sit in the same stage as well (one of them fails).

CS-Player-2: A player must not attempt to sit in the p+1 game if the players from the previous p-game have not yet gotten up from their chairs.

Notes:

1. The player who loses the stage p is the one who warns the rest that it is over.
2. In the p+1 stage there is one player and one less chair than in the p stage.

Exercise 2.2 (3 points): Extends the system of the previous part so that, at the end of the game, the players who have lost congratulate the winner. It uses only **binary semaphores** to implement the requested behaviour.

- Implements the **def congratulates(id: Int)** function of the MusicalChairs class. A player who has lost calls this function to congratulate the winner.
- Implements the **def waitCongratulations(): Unit** function of the MusicalChairs class. The player who has won calls this function repeatedly to accept congratulations from each other player and say thank you.

The synchronization conditions that must be implemented are:

CS-Player-3: A player who has lost waits for the winner to be available to congratulate it.

CS-Winner-1: The winner waits for a loser to congratulate it to thank it; the winner has to repeat this process until all the losers have congratulated him.

Uncomment in the **main** function the codes in which the players who have lost call the **congratulates** function and the winner successively calls **waitCongratulations** to accept them.

A possible execution of the game of chairs with 6 players and 5 chairs is shown below. The last lines with the congratulations correspond to the second part of the exercise:

```
Thread-1: Iteration 0: Player 1 is sat!! :-))
Thread-4: Iteration 0: Player 4 is sat!! :-))
Thread-3: Iteration 0: Player 3 is sat!! :-))
Thread-2: Iteration 0: Player 2 is sat!! :-))
```

```

Thread-5: Iteration 0: Player 5 is sat!! :-))
Thread-0: Iteration 0: Player 0 loses :-( Exits from the game
Thread-1: Player 1 exits from the room. 4
Thread-4: Player 4 exits from the room. 3
Thread-3: Player 3 exits from the room. 2
Thread-2: Player 2 exits from the room. 1
Thread-5: Player 5 exits from the room. 0
Thread-5: -----End iteration 0-----
Thread-5: Iteration 1: Player 5 is sat!! :-))
Thread-2: Iteration 1: Player 2 is sat!! :-))
Thread-4: Iteration 1: Player 4 is sat!! :-))
Thread-3: Iteration 1: Player 3 is sat!! :-))
Thread-1: Iteration 1: Player 1 loses :-( Exits from the game
Thread-5: Player 5 exits from the room. 3
Thread-2: Player 2 exits from the room. 2
Thread-4: Player 4 exits from the room. 1
Thread-3: Player 3 exits from the room. 0
Thread-3: -----End iteration 1-----
Thread-5: Iteration 2: Player 5 is sat!! :-))
Thread-2: Iteration 2: Player 2 is sat!! :-))
Thread-4: Iteration 2: Player 4 is sat!! :-))
Thread-3: Iteration 2: Player 3 loses :-( Exits from the game
Thread-5: Player 5 exits from the room. 2
Thread-2: Player 2 exits from the room. 1
Thread-4: Player 4 exits from the room. 0
Thread-4: -----End iteration 2-----
Thread-4: Iteration 3: Player 4 is sat!! :-))
Thread-5: Iteration 3: Player 5 is sat!! :-))
Thread-2: Iteration 3: Player 2 loses :-( Exits from the game
Thread-4: Player 4 exits from the room. 1
Thread-5: Player 5 exits from the room. 0
Thread-5: -----End iteration 3-----
Thread-5: Iteration 4: Player 5 is sat!! :-))
Thread-4: Iteration 4: Player 4 loses :-( Exits from the game
Thread-5: Player 5 exits from the room. 0
Thread-5: -----End iteration 4-----
Thread-5: Player 5 is the winner!!!! Waiting for other players' congratulations
Thread-0: Congratulations!!!
Thread-5: Many thanks...
Thread-1: Congratulations!!!
Thread-5: Many thanks...
Thread-3: Congratulations!!!
Thread-5: Many thanks...
Thread-2: Congratulations!!!
Thread-5: Many thanks...
Thread-4: Congratulations!!!
Thread-5: Many thanks...

```

IMPORTANT NOTE: You must remember that an output similar to the previous one does not imply that your implemented code is OK. To increase the variability of the execution you can run your solution several times, with different numbers of players or by modifying or removing the sleeps from the threads.

PART 3: Synchronizing with Monitors

Exercise 3: Implement the two parts of the Musical Chairs game described in part 2 using **synchronized methods or locks**. The points assigned to each part is the same than in exercises 2.1 and 2.2.

